

LAPORAN TUGAS BESAR
IF2224 – TEORI BAHASA FORMAL DAN OTOMATA



K01 - MLB

Mikhael Benrael Tampubolon	13524009
Muhammad Iqbal Raihan	13524011
Moh Hafizh Irham Perdana	13524025
Muhammad Akmal	13524099

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025

DAFTAR ISI

DAFTAR ISI.....	2
BAB 1.....	4
1.1. Intermediate Code Generation.....	4
1.1.1. Decorated AST sebagai Sumber Kebenaran.....	4
1.1.2. Three-Address Code (TAC).....	5
1.1.3. Translasi Control Flow.....	5
1.2. Arsitektur Interpreter: Stack Machines.....	6
1.2.1. Siklus Eksekusi (Fetch-Decode-Execute).....	6
1.2.2. Memori Eksekusi: The Stack.....	7
1.2.3. Daftar Instruksi.....	7
1.3. Arsitektur Front-End dan Back-End.....	9
BAB 2.....	10
PERANCANGAN DAN IMPLEMENTASI.....	10
2.1. Arsitektur Pipeline.....	10
2.2. Perancangan Representasi Instruksi.....	10
2.3. Perancangan Code Generator.....	11
2.3.1. Struktur Program dan Backpatching.....	11
2.3.2. Translasi Pernyataan.....	11
2.3.3. Translasi Ekspresi dan Deklarasi Subprogram.....	12
2.4. Perancangan Interpreter.....	12
2.4.1. Akses Memori dan Parameter.....	12
2.4.2. Keamanan Runtime.....	13
BAB 3.....	14
PENGUJIAN.....	14
3.1. Hasil Pengujian.....	14
A. Pengujian ke-1.....	14
B. Pengujian ke-2.....	14
C. Pengujian ke-3.....	15
D. Pengujian ke-4.....	15
E. Pengujian ke-5.....	16
F. Pengujian ke-6.....	16
G. Pengujian ke-7.....	17
H. Pengujian ke-8.....	17
BAB 4	
KESIMPULAN & SARAN.....	18
4.1. Kesimpulan.....	18
4.2. Saran.....	19
BAB 5.....	20
LAMPIRAN.....	20
1. Link Release Repository Github.....	20
2. Pembagian Tugas.....	20

BAB 1

TEORI DASAR

1.1. Intermediate Code Generation

Intermediate Code Generation merupakan fase keempat dalam proses kompilasi, yang berfungsi sebagai jembatan antara fase *front-end* (analisis leksikal, sintaksis, dan semantik) dengan fase *back-end* (eksekusi). Pada tahap ini, representasi program berupa *Decorated AST* yang telah tervalidasi secara sintaksis maupun semantik diubah menjadi serangkaian instruksi linier yang lebih sederhana dan mendekati bahasa mesin, namun masih cukup mudah dibaca oleh manusia.

Hasil dari tahap ini disebut *Intermediate Code* (kode perantara). Penggunaan kode perantara memberikan beberapa keuntungan penting. Pertama, ia memisahkan tanggung jawab antara *front-end* dan *back-end* sehingga keduanya dapat dikembangkan secara independen. Kedua, kode perantara bersifat *platform-independent*, artinya satu representasi yang sama dapat dieksekusi di berbagai arsitektur mesin yang berbeda hanya dengan mengubah *back-end*-nya. Ketiga, kode perantara yang berbentuk linier jauh lebih mudah dioptimasi dibandingkan struktur pohon yang bersarang.

1.1.1. Decorated AST sebagai Sumber Kebenaran

Sebelum kode perantara dapat dibangkitkan, diperlukan sumber kebenaran yang mutlak berupa *Decorated AST* (Abstract Syntax Tree yang telah didekorasi). *Decorated AST* adalah hasil akhir dari fase *Semantic Analysis* di mana setiap *node* pada AST telah dianotasi dengan informasi semantik tambahan, meliputi:

1. **Tipe Data:** Menyatakan tipe dari ekspresi pada *node* tersebut (misalnya integer, real, boolean).
2. **Referensi Tabel Simbol:** Menyatakan *pointer* ke entri variabel di dalam tabel simbol, sehingga generator mengetahui ruang lingkup (*scope*) dan alamat memori dari setiap variabel.

Informasi ini sangat krusial karena tanpa anotasi tipe data, generator tidak dapat membedakan apakah operasi $a + b$ merupakan penjumlahan bilangan bulat, penjumlahan bilangan riil, atau penggabungan string. Dengan *Decorated AST*, keputusan tersebut dapat dibuat secara otomatis dan tepat.

1.1.2. Three-Address Code (TAC)

Format kode perantara yang paling umum digunakan di industri adalah *Three-Address Code* (TAC). Dinamakan demikian karena setiap instruksi dirancang sangat sederhana, di mana maksimal hanya melibatkan tiga alamat memori — biasanya dua *operand* sumber dan satu lokasi tujuan. Struktur dasar TAC adalah:

$\text{hasil} := \text{operand1 operator operand2}$

Untuk menangani ekspresi kompleks yang bersarang, TAC menggunakan variabel sementara (biasanya dinotasikan $t1$, $t2$, $t3$, dst.). Generator akan meratakan (*flatten*) pohon ekspresi dari cabang terdalam menuju akar, menghasilkan instruksi linier secara bertahap. Sebagai contoh, ekspresi $x := (a + b) * (c - d)$ akan diuraikan menjadi:

- $t1 := a + b$
- $t2 := c - d$
- $t3 := t1 * t2$
- $x := t3$

1.1.3. Translasi Control Flow

Tantangan terbesar dalam pembangunan kode perantara adalah meratakan struktur kontrol (*control flow*) yang bersarang, seperti IF-ELSE dan WHILE, menjadi daftar instruksi linier. Hal ini dicapai dengan kombinasi tiga konstruk:

1. **Label:** Penanda statis yang menunjukkan posisi baris tertentu dalam TAC (contoh: `L_ELSE;`, `L_END;`).

2. **Jump Bersyarat (JPC):** Melompat ke suatu *label* hanya jika kondisi yang dievaluasi bernilai *false*.
3. **Jump Mutlak (JMP):** Melompat secara langsung ke suatu *label* tanpa memedulikan kondisi apapun.

Pola translasi untuk blok IF-ELSE adalah sebagai berikut: kondisi dievaluasi terlebih dahulu, lalu instruksi JPC ke L_ELSE dipasang. Jika kondisi terpenuhi, blok THEN dieksekusi dan diakhiri dengan JMP ke L_END. Jika tidak terpenuhi, eksekusi melompat ke L_ELSE untuk menjalankan blok ELSE.

Untuk perulangan WHILE, sebuah *label* awal (L_START) dipasang sebelum evaluasi kondisi. Jika kondisi tidak terpenuhi, JPC melompat ke L_END. Jika terpenuhi, blok *body* dieksekusi dan diakhiri dengan JMP kembali ke L_START untuk mengevaluasi ulang kondisi. Pendekatan yang digunakan adalah JPC (*Jump if Condition false*), yang memanfaatkan konsep *fall-through*, yaitu ketika kondisi bernilai benar, eksekusi langsung berlanjut ke instruksi berikutnya tanpa lompatan, sehingga alur kontrol lebih sederhana dan efisien.

1.2. Arsitektur Interpreter: Stack Machines

Setelah *Intermediate Code* berhasil dibangkitkan, tahap berikutnya adalah eksekusi. Tugas ini diemban oleh **Interpreter**, yang bertindak layaknya sebuah *Virtual Machine* (VM). Interpreter membaca instruksi TAC satu per satu dan menjalankannya secara nyata, menyimpan nilai variabel ke memori, mengevaluasi operasi matematika, mengelola alur percabangan, hingga mencetak output ke layar.

1.2.1. Siklus Eksekusi (Fetch-Decode-Execute)

Interpreter bekerja dalam sebuah siklus eksekusi yang terdiri dari tiga tahap berulang:

- **Fetch:** Membaca instruksi TAC yang saat ini ditunjuk oleh *Instruction Pointer* (IP).

- **Decode:** Mengurai instruksi tersebut untuk mengetahui jenis operasinya dan *operand*-nya.
- **Execute:** Menjalankan aksi nyata sesuai instruksi, seperti perhitungan aritmatika, penyimpanan nilai ke memori, atau lompatan ke baris lain.

Setelah tahap *Execute* selesai, IP bergerak maju ke instruksi berikutnya ($IP := IP + 1$), kecuali instruksi tersebut adalah **JMP** atau **JPC** yang menyebabkan IP melompat ke nomor baris target. Siklus ini terus berulang hingga instruksi **RET** ditemukan.

1.2.2. Memori Eksekusi: The Stack

Struktur memori yang paling vital dalam arsitektur interpreter adalah **Stack** (tumpukan). Stack bekerja berdasarkan prinsip **LIFO (Last In, First Out)** — data yang terakhir dimasukkan (*push*) adalah data pertama yang diambil (*pop*). Alasan penggunaan Stack bukan sekadar *array* biasa berkaitan erat dengan pengelolaan *scope* dan pemanggilan fungsi/prosedur. Setiap kali program memasuki sebuah blok kode baru atau memanggil fungsi, Interpreter membuat sebuah kotak memori baru di dalam Stack yang disebut **Stack Frame**.

Sebuah *Stack Frame* terdiri dari tiga bagian yang selalu ada:

- **Static Link:** Menunjuk ke *frame* dari *scope* leksikal yang melingkupi fungsi tersebut, digunakan untuk mengakses variabel dari *scope* luar.
- **Dynamic Link:** Menunjuk ke *frame* dari pemanggil (*caller*), digunakan untuk memulihkan kondisi Stack setelah fungsi selesai.
- **Return Address:** Menyimpan nomor baris instruksi TAC tempat eksekusi harus dilanjutkan setelah fungsi mengembalikan nilai.

Setelah ketiga bagian tersebut, variabel-variabel lokal milik fungsi dialokasikan secara berurutan. Saat fungsi selesai dieksekusi, *frame* tersebut di-*pop* dari Stack dan semua variabel lokalnya otomatis musnah, sehingga VM kembali fokus pada *frame* pemanggil di bawahnya.

1.2.3. Daftar Instruksi

Berikut adalah instruksi-instruksi yang digunakan dalam Intermediate Code beserta cara kerjanya pada Stack Machine:

Instruksi	Operasi	Keterangan
LIT v	Load Literal	Memasukkan nilai literal v ke dalam stack
LOD a	Load Value	Memuat nilai dari alamat memori a ke stack
STO a	Store Value	Mengambil nilai dari puncak stack dan menyimpannya ke alamat a
CAL l	Call	Memanggil fungsi atau prosedur di baris l
INT m	Initiate Memory	Mengalokasikan memori sebesar m untuk satu <i>stack frame</i>
JMP l	Unconditional Jump	Melompat ke baris l tanpa kondisi apapun
JPC l	Conditional Jump	Melompat ke baris l bila kondisi di puncak stack bernilai <i>false</i>
OPR o	Operation	Memanggil operasi o (termasuk aritmatika, komparasi, dan I/O)
RET	Return	Keluar dari fungsi atau prosedur, memulihkan <i>frame</i> sebelumnya

Operasi-operasi yang tersedia untuk instruksi OPR mencakup operasi aritmatika (ADD, SUB, MUL, DIV, MOD, NEG), operasi perbandingan (EQL, NEQ, LSS, GEQ, GTR, LEQ), serta operasi I/O (WRT untuk menulis output dan WRTLN untuk menulis output dengan baris baru).

1.3. Arsitektur Front-End dan Back-End

Dalam perancangan *compiler* dan *interpreter* modern, sistem umumnya dibagi menjadi dua fase arsitektur utama: **Front-end** dan **Back-end**. *Front-end* mencakup seluruh tahapan yang berhadapan langsung dengan *source code* yang telah dibuat, yaitu Milestone I (Lexical Analysis), Milestone II (Syntax Analysis), dan Milestone III (Semantic Analysis). Hasil akhir dari *front-end* adalah struktur data murni berupa *Decorated AST* yang sudah terbebas dari teks mentah.

Back-end sama sekali tidak peduli dengan sintaksis bahasa Arion. Tugasnya berfokus murni pada dua hal:

1. **Sintesis:** Mengonversi *Decorated AST* menjadi instruksi operasional standar (*Intermediate Code*).
2. **Eksekusi:** Mengelola memori dan menjalankan instruksi tersebut pada mesin virtual (*Interpreter*).

Pemisahan arsitektur yang tegas antara *front-end* dan *back-end* ini sangat penting karena memberikan fleksibilitas tinggi. Jika suatu saat bahasa Arion ingin diperbarui dengan sintaks yang berbeda, hanya *front-end*-nya yang perlu diubah, selama ia tetap menghasilkan *Decorated AST* dengan format yang sama, maka *back-end* dapat mengeksekusinya tanpa modifikasi apapun.

BAB 2

PERANCANGAN DAN IMPLEMENTASI

2.1. Arsitektur Pipeline

Program arion mengintegrasikan seluruh fase kompilasi ke dalam satu berkas executable tunggal. Alur eksekusi penuh ketika dijalankan tanpa opsi dump dimulai dengan tahap analisis leksikal oleh Lexer (`src/lexer/`) untuk melakukan tokenisasi berkas sumber. Hasil tokenisasi tersebut kemudian diproses oleh Parser (`src/parser/`) untuk membangun pohon analisis (parse tree). Selanjutnya, Semantic Analyzer (`src/semantic/`) melakukan analisis semantik untuk menghasilkan AST beranotasi (Decorated AST) sekaligus membangun tabel simbol. Berdasarkan pohon sintaksis yang telah beranotasi tersebut, Code Generator (`src/ir/code_generator.hpp`) melakukan translasi untuk menghasilkan kode perantara (TAC). Alur eksekusi diakhiri oleh Interpreter (`src/vm/interpreter.hpp`) yang mengeksekusi instruksi-instruksi TAC pada mesin stack virtual.

Proses pembangkitan kode perantara dan interpretasi hanya akan dieksekusi apabila tahap analisis semantik berhasil diselesaikan tanpa kesalahan, serta program tidak dipanggil dengan argumen opsi dump atau dump-all. Penggunaan mode dump akan membatasi alur kompilasi hanya sampai pada penulisan berkas keluaran `.token`, `.ptree`, `.ast`, dan `.sym.*` untuk mendukung kompatibilitas pengujian modul pada milestone sebelumnya.

2.2. Perancangan Representasi Instruksi

Modul `src/ir/instruction.hpp` mendefinisikan enumerasi `OpCode` dan struktur `Instruction` yang terdiri atas empat field utama:

- **op**
Menyatakan jenis operasi (seperti LIT, LOD, hingga STX).
- **l**
Menyimpan selisih level leksikal (level difference) untuk alokasi variabel dalam cakupan leksikal.
- **a**
Merepresentasikan alamat, nilai literal, indeks lompatan, atau kode operasi OPR tertentu.
- **aux**
Berfungsi sebagai operand tambahan, terutama digunakan untuk menyimpan ukuran array pada instruksi LDX dan STX.

Representasi tekstual dari kode perantara yang dihasilkan mengikuti pola terstruktur `<baris><opcode><l><a>[<aux>]`. Format linier ini dirancang agar mudah dibaca

untuk keperluan verifikasi manual serta memfasilitasi pencocokan otomatis oleh sistem pengujian (*test harness*).

2.3. Perancangan Code Generator

Kelas `CodeGenerator` memanfaatkan pola desain Visitor dengan mengimplementasikan antarmuka `ast::ASTVisitor` pada objek *Decorated AST*. Melalui pendekatan ini, setiap metode kunjungan (*visit*) pada simpul AST tertentu secara terarah menghasilkan urutan instruksi TAC yang merepresentasikan semantik dari simpul tersebut.

2.3.1. Struktur Program dan Backpatching

Saat memproses simpul utama `ProgramNode`, generator menempatkan instruksi lompatan awal (**JMP**) ke badan program utama. Tindakan ini dilakukan agar deklarasi seluruh prosedur dan fungsi dapat ditulis terlebih dahulu di bagian awal kode perantara, diikuti dengan pengisian kembali (*backpatching*) alamat lompatan yang sesuai. Bingkai stack utama dialokasikan menggunakan instruksi **INT** dengan ukuran $3 + vsze$, yang mencakup tiga sel untuk kebutuhan tautan kontrol (*links*) ditambah ruang bagi variabel lokal.

2.3.2. Translasi Pernyataan

Translasi berbagai jenis pernyataan diimplementasikan dengan memetakan setiap konstruksi sintaksis ke instruksi TAC yang sesuai.

Pada pernyataan penugasan (*Assign*), generator terlebih dahulu mengevaluasi ekspresi di ruas kanan sebelum menghasilkan instruksi **STO**. Khusus untuk elemen array, indeks dinormalisasi melalui pengurangan terhadap batas bawah (`index - low`) sebelum diakses menggunakan **STX**, sedangkan untuk struktur *record*, offset field ditambahkan langsung ke alamat basisnya.

Struktur kontrol alur diterjemahkan menggunakan kombinasi instruksi percabangan dan lompatan sebagai berikut:

- **If-Else**
Menghasilkan instruksi **JPC** untuk melompati blok *then* menuju blok *else* jika kondisi tidak terpenuhi, serta instruksi **JMP** di akhir blok *then* untuk menghindari eksekusi blok *else*.
- **While**
Menetapkan label awal untuk mengevaluasi kondisi, menghasilkan **JPC** untuk keluar dari perulangan jika kondisi bernilai salah, dan menempatkan **JMP** di akhir badan perulangan untuk kembali ke evaluasi kondisi.
- **Repeat-Until**
Badan perulangan dieksekusi terlebih dahulu sebelum kondisi di akhir

dievaluasi; instruksi JPC akan mengarahkan alur kembali ke awal perulangan selama kondisi tersebut belum terpenuhi.

- **For**
Menginisialisasi variabel iterator, membandingkannya dengan nilai batas (baik menaik menggunakan `to` maupun menurun dengan `downto`), mengeksekusi badan perulangan, melakukan pembaruan nilai iterator (increment/decrement), lalu melakukan lompatan kembali ke awal pemeriksaan.

Terakhir, pemanggilan prosedur dan fungsi (ProcCall / FuncCall) diterjemahkan dengan mengevaluasi argumen-argumen secara berurutan dan mendorong nilainya ke stack, yang kemudian diikuti oleh pemanggilan instruksi **CAL**. Adapun pemanggilan fungsi atau prosedur bawaan seperti `write` dan `writeln` langsung dipetakan ke instruksi **OPR** dengan sub-kode masing-masing 13 dan 14.

2.3.3. Translasi Ekspresi dan Deklarasi Subprogram

Evaluasi ekspresi biner dilakukan dengan memetakan operator token secara langsung ke kode operasi **OPR** yang bersesuaian. Sementara itu, akses terhadap elemen array diselesaikan melalui instruksi **LDX** dengan menghitung indeks relatif terhadap nilai batas bawah array yang tersimpan dalam atribut `atab.low`. Pada deklarasi prosedur, alamat awal eksekusi dicatat pada `eld entry.adr` di tabel simbol, sedangkan bagian badan subprogram dibungkus secara berurutan oleh instruksi alokasi **INT**, blok pernyataan, dan diakhiri dengan instruksi **RET**. Khusus untuk subprogram bertipe fungsi, nilai kembalian akan diletakkan di puncak stack sebelum instruksi **RET** dijalankan.

2.4. Perancangan Interpreter

Komponen Interpreter yang diimplementasikan pada `src/vm/interpreter.hpp` mensimulasikan jalannya mesin stack virtual dengan mengelola tiga register utama, yaitu `pc` (*program counter*), `b` (*base pointer*), dan `t` (*stack pointer*). Ruang memori data disimulasikan menggunakan array linear `s` yang dialokasikan sebesar 10.000 sel.

2.4.1. Akses Memori dan Parameter

Untuk mendukung ruang lingkup variabel bertingkat, fungsi internal `getBase(1)` digunakan untuk menelusuri rantai *static link* sebanyak 1 kali. Instruksi **LOD** dan **STO** dengan alamat offset bernilai negatif digunakan untuk mengakses parameter fungsi/prosedur secara relatif terhadap bingkai pemanggil (`b+a`). Saat pemanggilan subprogram terjadi, instruksi **CAL** secara otomatis menyimpan informasi *static link*,

dynamic link, dan alamat kembali (*return address*) pada *stack* sebelum mengalihkan *program counter* ke lokasi tujuan.

2.4.2. Keamanan Runtime

Untuk menjamin keamanan eksekusi pada tingkat *runtime*, interpreter melakukan serangkaian validasi kesalahan dinamis dan mencetak pesan kesalahan yang relevan ke aliran **stderr**. Beberapa jenis validasi kesalahan dinamis yang ditangani oleh interpreter meliputi:

- **DivisionByZeroError**

Terjadi jika terdeteksi operasi pembagian atau operasi modulo dengan pembagi bernilai nol.

- **IndexOutOfBoundsException**

Terjadi jika indeks array yang diakses berada di luar rentang valid yang dialokasikan (**[0, aux)**).

- **StackUnderflowError / StackOverflowError**

Terjadi jika terdapat pelanggaran batas kapasitas *stack* (underflow atau overflow).

- **InvalidJumpTarget**

Terjadi jika instruksi mencoba melompat ke baris instruksi di luar batas kode program yang tersedia.

BAB 3

PENGUJIAN

3.1. Hasil Pengujian

A. Pengujian ke-1

test/milestone-4/text_keyword.txt	
Input	Output
<pre> program TestKeywords; var x , y: integer; begin x := 10; if x > 5 then y := x * 2 else y := x div 2; writeln (y); while y > 0 do begin y := y - 1; writeln (y); end; end. </pre>	<pre> 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0 </pre>
Keterangan: Sesuai	

B. Pengujian ke-2

test/milestone-4/if_else.txt	
Input	Output
<pre> program StressTest; var x, y, z : integer; begin x := 5; y := 2; z := x * y + 10 div 2; </pre>	<pre> 12 </pre>

<pre> if z >= 15 then z := z - 3 else z := z + 3; while z > 10 do begin writeln(z); z := z - 2; end; end. </pre>	
Keterangan: Sesuai	

C. Pengujian ke-3

test/milestone-4/exec_for.pas	
Input	Output
<pre> program ExecFor; var i: integer; begin for i := 1 to 3 do begin writeln(i); end; end. </pre>	<pre> 1 2 3 </pre>
Keterangan: Sesuai	

D. Pengujian ke-4

test/milestone-4/exec_array.pas	
Input	Output
<pre> program ExecArray; var data: array[1..3] of integer; begin data[1] := 10; data[2] := 20; data[3] := 30; writeln(data[2]); end. </pre>	<pre> 20 </pre>
Keterangan: Sesuai	

E. Pengujian ke-5

test/milestone-4/exec_func_call.pas	
Input	Output
<pre>program ExecFuncCall; var r: integer; function Add(a, b: integer): integer; begin Add := a + b; end; begin r := Add(3, 4); writeln(r); end.</pre>	7
Keterangan: Sesuai	

F. Pengujian ke-6

test/milestone-4/exec_proc.pas	
Input	Output
<pre>program ExecProc; procedure SayHello; begin writeln(42); end; begin SayHello(); end.</pre>	42
Keterangan: Sesuai	

G. Pengujian ke-7

test/milestone-4/neg_array_oob.pas	
Input	Output
<pre>program NegArrayOob; var data: array[1..3] of integer; begin writeln(data[10]); end.</pre>	IndexOutOfRangeException
Keterangan: Sesuai	

H. Pengujian ke-8

test/milestone-4/neg_div_zero.pas	
Input	Output
<pre>program NegDivZero; var x: integer; begin x := 1 div 0; end.</pre>	DivisionByZeroError
Keterangan: Sesuai	

BAB 4

KESIMPULAN & SARAN

4.1. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan pada Milestone 4, dapat ditarik beberapa kesimpulan penting mengenai pengembangan kompilator Arion.

Pertama, modul Intermediate Code Generation (ICG) yang memanfaatkan pola Visitor pada Decorated AST telah berhasil menerjemahkan berbagai konstruksi sintaksis seperti struktur kontrol, ekspresi, array, record, dan subprogram menjadi kode perantara linear (TAC) dengan mekanisme backpatching yang konsisten.

Kedua, arsitektur stack machine yang dirancang dengan sekumpulan instruksi dasar (LIT, LOD, STO, CAL, dan OPR) terbukti mampu mengeksekusi program Pascal-subset Arion secara menyeluruh, termasuk penanganan prosedur masukan-keluaran bawaan seperti write dan writeln.

Selanjutnya, integrasi seluruh tahapan kompilasi di dalam main.cpp memungkinkan pemisahan yang jelas antara mode analisis (-dump) dan mode eksekusi penuh, sehingga kepatuhan terhadap spesifikasi tugas besar pada milestone sebelumnya tetap terjaga tanpa harus menjalankan mesin virtual.

Validasi kebenaran translasi dan eksekusi ini diperkuat oleh keberhasilan rangkaian pengujian otomatis pada skrip test/run_all.ps1, yang mencakup verifikasi kasus positif dan penanganan kesalahan waktu berjalan (runtime error) seperti pembagian dengan nol dan akses indeks di luar batas.

Terakhir, struktur tabel simbol yang memanfaatkan representasi display dan proses nalisasi alamat parameter terbukti krusial dalam mendukung pencarian simbol pada cakupan leksikal dinamis, sehingga instruksi pemuatan (LOD) dan penyimpanan (STO) dapat bekerja dengan selisih tingkat cakupan (level difference) yang tepat.

4.2. Saran

Untuk pengembangan dan penyempurnaan kompilator Arion di masa mendatang, terdapat beberapa saran perbaikan yang dapat diimplementasikan.

Dari segi arsitektur kode, pemisahan implementasi CodeGenerator dan Interpreter ke dalam berkas .cpp terpisah yang terintegrasi melalui CMakeLists.txt sangat disarankan untuk mempercepat waktu kompilasi main.cpp sekaligus mempermudah pengujian unit (unit testing) pada setiap modul.

Selain itu, kapabilitas eksekusi mesin virtual dapat ditingkatkan dengan menambahkan dukungan tipe data riil (real) secara penuh, mengingat operasi stack saat ini masih terbatas pada representasi integer, serta mengimplementasikan prosedur pembacaan interaktif seperti read dan readln pada lingkungan runtime.

Penyempurnaan pada aspek optimasi dan penanganan tipe data juga krusial untuk dilakukan. Sistem masukan-keluaran string sebaiknya diperluas agar tidak sekadar mendorong deretan kode ASCII ke stack, melainkan mendukung pencetakan string sebagai entitas tunggal melalui penambahan operasi OPR yang spesifik.

Di samping itu, penerapan optimasi kode perantara yang sederhana seperti optimasi lubang intip (peephole optimization) atau eliminasi pasangan instruksi LIT/STO yang redundan dapat meningkatkan efisiensi eksekusi dan keterbacaan TAC yang dihasilkan.

Terakhir, dokumentasi formal mengenai konvensi alamat parameter bernilai negatif dan tata letak stack frame perlu disusun dalam spesifikasi internal proyek guna menjamin konsistensi pemeliharaan kode antara modul analisis semantik dan modul pembangkit kode di masa yang akan datang.

BAB 5

LAMPIRAN

- 1. Link Release Repository Github.**
- 2. Pembagian Tugas.**

NIM	Nama	Pembagian Tugas	Persentase
13524009	Mikhael Benrael Tampubolon	Penulisan Laporan, Implementasi Code Generator, Interpreter	25%
13524011	Muhammad Iqbal Raihan	Penulisan Laporan, Implementasi Interpreter & Stack Machine	25%
13524025	Moh Hafizh Irham Perdana	Penulisan Laporan	25%
13524099	Muhammad Akmal	Implementasi Instruction Set & integrasi antar modul	25%